Global Knowledge®

Angular Fundamentals
Korte herhaling dag #1

Peter Kassenaar –
info@kassenaar.com

WORLDWIDE LOCATIONS

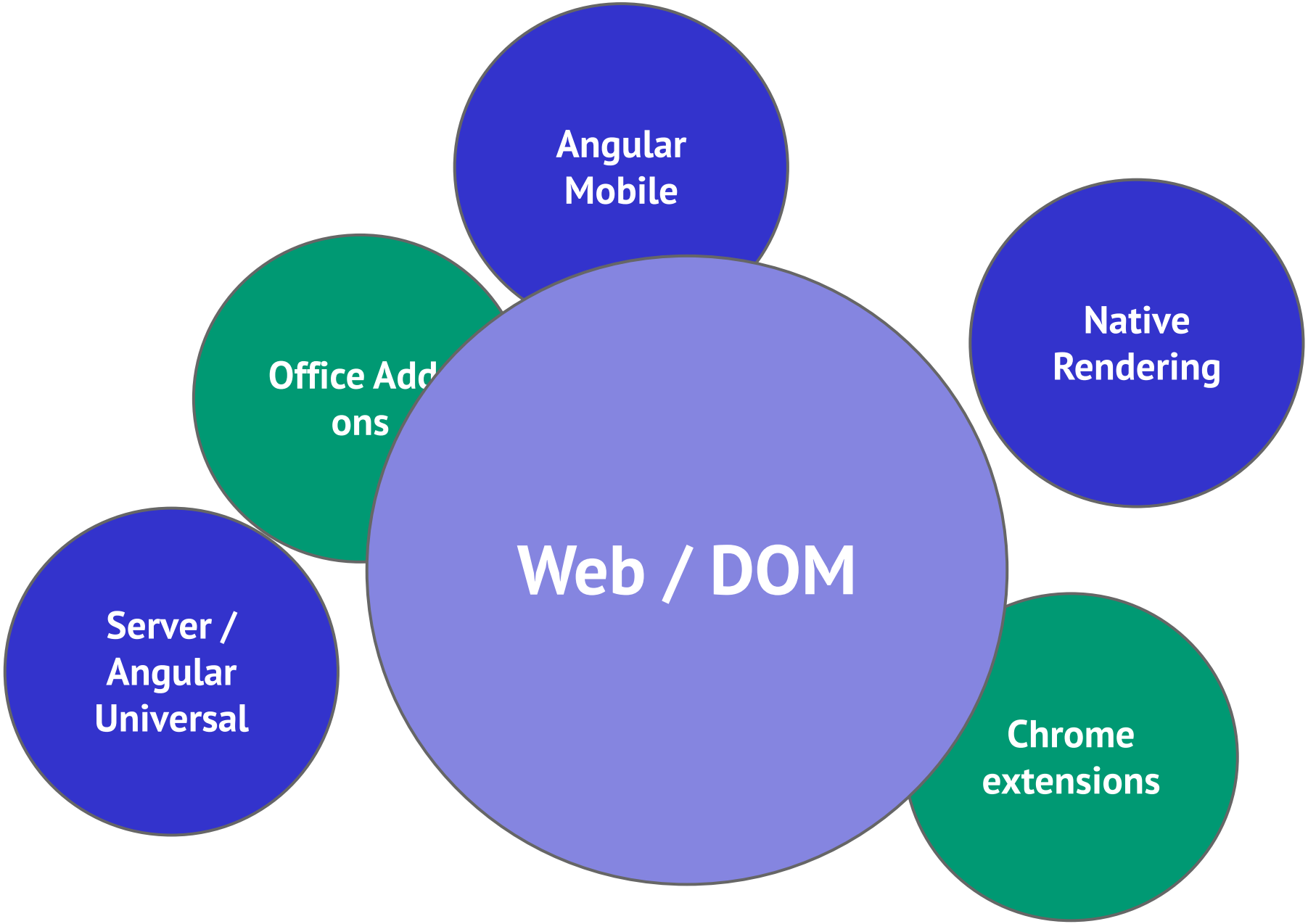
BELGIUM CANADA COLOMBIA DENMARK EGYPT FRANCE IRELAND JAPAN KOREA MALAYSIA MEXICO NETHERLANDS NORWAY QATAR
SAUDI ARABIA SINGAPORE SPAIN SWEDEN UNITED ARAB EMIRATES UNITED KINGDOM UNITED STATES OF AMERICA

Agenda

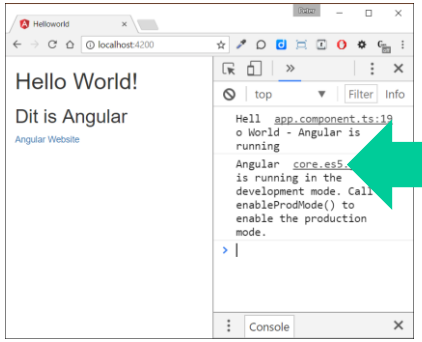
- Day 1 – Intro & Data binding
  - Theorie - Introductie & geschiedenis
    - Angular vs Vue, React
  - Hello World in Angular –boilerplate-code
  - Concepts, context & architecture
  - Angular CLI, TypeScript
  - Components
  - Data binding

Framework to Platform

	Scaffolding	Code completion & Refactoring	Debugging
Tooling	Angular CLI	Language Services	DevTools
Libraries	Material 2	Mobile	Universal
	Compile	Change Detection	Renderer
Core	Components & Dependency Injection	Decorators	Zones


Angular Mobile
Office Add ons
Native Rendering
Web / DOM
Server / Angular Universal
Chrome extensions

Existing Angular apps



main.ts / bootstrapper

ngModule / root module

AppComponent

Services

3rd party modules

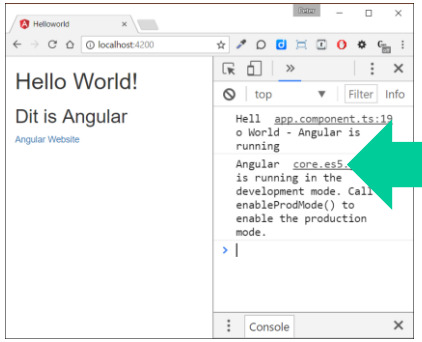
Other modules

Other components

Other components

Other components

Modern Angular apps



main.ts / bootstrapper

Services

AppComponent

3rd party libs

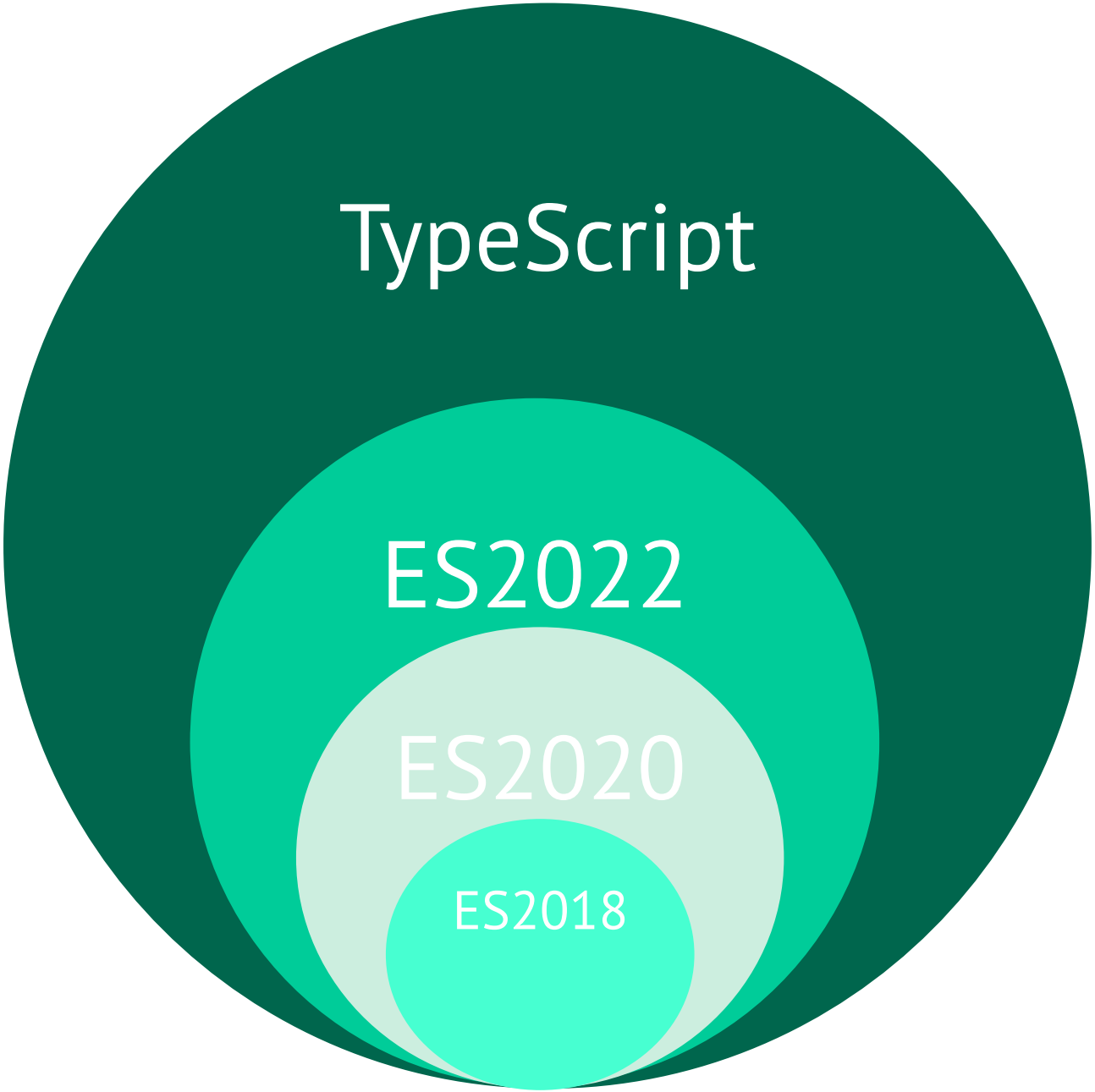
Other modules

Other components

Other components

Other components

(Note: no `ngModule` anymore, as this is now handled by standalone components)


TypeScript
ES2022
ES2020
ES2018

Boilerplate code - stappenplan

1. Set up environment, boilerplate & libraries
   - `npm install`
2. Schrijf Angular Root Component & Module voor de app
3. Bootstrap
4. Schrijf HTML-pagina (`index.html`)
5. Run applicatie
   - `npm start`

Components

- `@Component` decorator
- View
- Controller / Class

```
import { Component, OnInit } from '@angular/core';

@Component({
    selector: 'app-customer',
    templateUrl: './customer.component.html',
    styleUrls: ['./customer.component.css']
})
export class CustomerComponent implements OnInit {

    constructor() {}

    ngOnInit() {}
}
```

Angular CLI

Take the 2024 Angular Developer Survey today!

v18

Ctrl K

Docs

Tutorials

Playground

Reference

Angular CLI

- Overview
- Local set-up
- Building Angular apps
- Serving Angular apps for development
- Deployment
- End-to-End Testing
- Migrating to new build system
- Build environments
- Angular CLI builders
- Generating code using schematics
- Authoring schematics
- Schematics for libraries
- Template type checking
- Ahead-of-time (AOT) compilation
- AOT metadata errors

Developer Tools › Angular CLI

The Angular CLI

The Angular CLI is a command-line interface tool which allows you to scaffold, develop, test, deploy, and maintain Angular applications directly from a command shell.

Angular CLI is published on npm as the `@angular/cli` package and includes a binary named `ng`. Commands invoking `ng` are using the Angular CLI.

Try Angular without local setup

If you are new to Angular, you might want to start with Try it now!, which introduces the essentials of Angular in the context of a ready-made basic online store app for you to examine and modify. This standalone tutorial takes advantage of the interactive StackBlitz environment for online development. You don't need to set up your local environment until you're ready.

Getting Started

Install Angular CLI to create and build your first app.

Get Started

Command Reference

Discover CLI commands to make you more productive with Angular.

Learn More

Schematics

Create and run schematics to generate and modify source files in your application automatically.

Builders

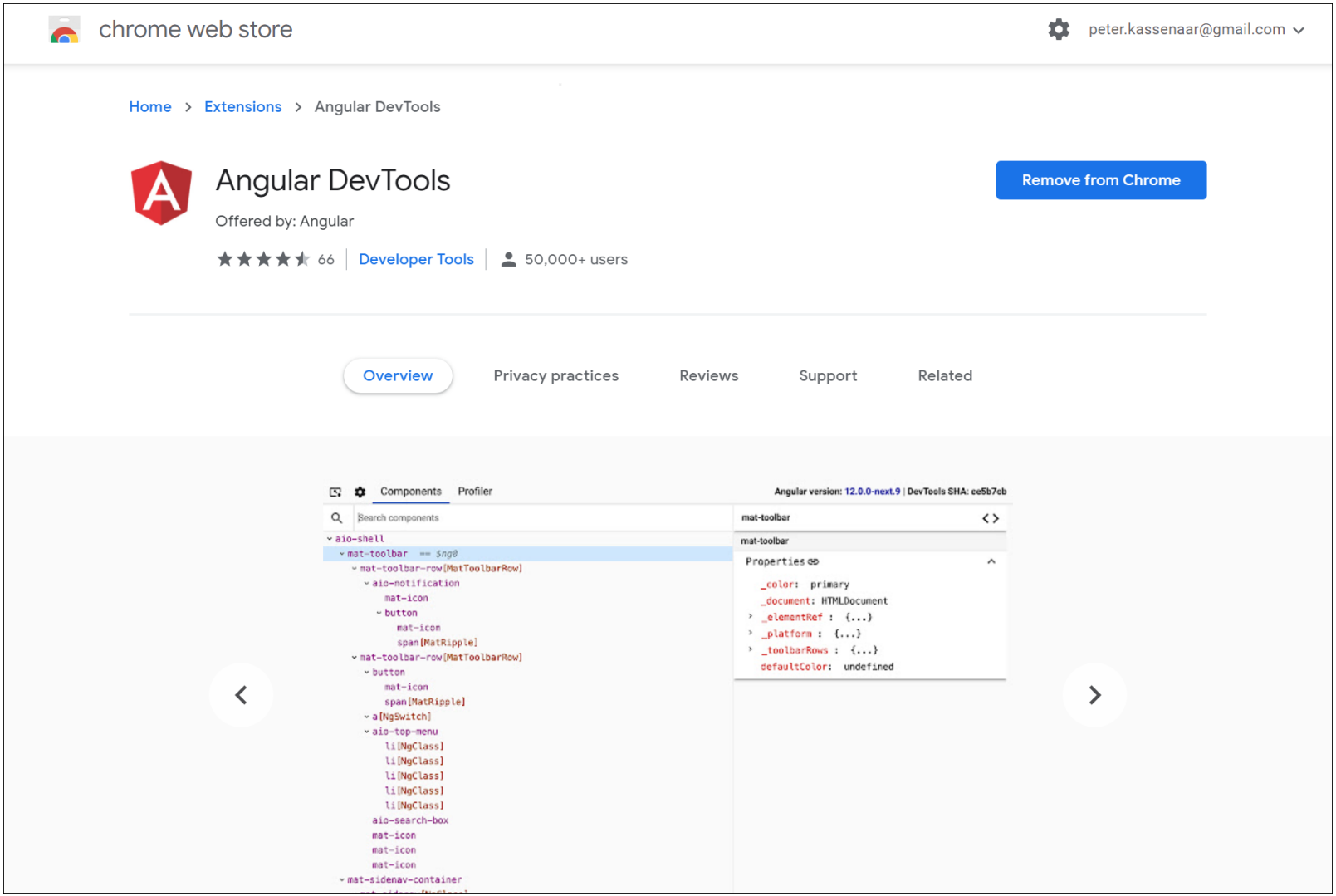
Create and run builders to perform complex transformations from your source code to generated build outputs.

On this page

- CLI command-language syntax
  - Boolean options
  - Array options
  - Relative paths

https://angular.dev/tools/cli

Debugging/Analyse tool – Chrome DevTools



https://chrome.google.com/webstore/detail/angular-devtools/ienfalfjdbdpebioblfackkekamfmbnh

Ng serve

- Vervult verschillende rollen (*kan* ook handmatig)
    1. TypeScript compilation
    2. (SASS compilation)
    3. Fire up webserver on http://localhost:4200
    4. Fire up Live Reload
    5. Analyzing and bundling of application by firing up WebPack
    6. Watch for changes and recompile, rebundle, etc.

Data binding

- Notatiewijze in HTML-views/templates
  1. Simple data binding `{{ … }}`
  2. Event binding `(…)="..."`
  3. One-way data binding `[…]="..."`
  4. Two-way data binding `[(ngModel)] = " …"`

Data doorgeven van View → Controller

1. Using `$event`

2. Using a Local Template Variabele `#NameVar`

3. Using `[(ngModel)]` (to be used in simple situations, mostly not on complex forms)

4. HostBinding/`@HostListener` (via @-decorators)

5. `@ViewChild('varName')` voor doorgeven DOM-element aan controller/class

Vragen?

Vandaag

- Belangrijk concept: Services
  - Static Services
  - Dependency Injection / `@Injectable()`
  - `HttpClientModule`,
  - RxJS / Observables,
  - `GET`-ing en `POST`-ing data
- Component Communication
  - `@Input(),@Output(), Service Bus`

Maar eerst – Style Guide / Best practices

Take the 2024 Angular Developer Survey today!

v18

Ctrl K

Docs

Tutorials

Playground

Reference

Introduction

What is Angular?

Installation

Essentials

Start coding!

In-depth Guides

Components

Templates

Directives

Dependency Injection

Signals

Routing

Forms

HTTP Client

Performance

Testing

Internationalization

Animations

Experimental features

Developer Tools

Angular CLI

Libraries

Best Practices

Angular coding style guide

Looking for an opinionated guide to Angular syntax, conventions, and application structure? Step right in. This style guide presents preferred conventions and, as importantly, explains why.

Style vocabulary

Each guideline describes either a good or bad practice, and all have a consistent presentation.

The wording of each guideline indicates how strong the recommendation is.

Do is one that should always be followed. *Always* might be a bit too strong of a word. Guidelines that literally should always be followed are extremely rare. On the other hand, you need a really unusual case for breaking a *Do* guideline.

Consider guidelines should generally be followed. If you fully understand the meaning behind the guideline and have a good reason to deviate, then do so. Aim to be consistent.

Avoid indicates something you should almost never do. Code examples to *avoid* have an unmistakable red header.

Why?
Gives reasons for following the previous recommendations.

File structure conventions

On this page

Style vocabulary
File structure conventions
Single responsibility
Rule of One
Naming
General Naming Guidelines
Separate file names with dots and dashes
Symbols and file names
Service names
Bootstrapping
Component selectors
Component custom prefix
Directive selectors
Directive custom prefix
Pipe names
Unit test file names
Application structure and NgModules
Overall structural guidelines
Folders-by-feature structure
App root module
Feature modules

https://angular.dev/style-guide

Woensdag 2 oktober – dag #3

- Vragen + herhaling Observables
- Vragen Component Communication
  - `@Input()`, `@Output()`
  - Custom Event Bus, `Subject<Type>()`
- Routing
  - Basic routing
  - Routing parameters
- Forms
  - Template Driven Forms
  - Model Driven Forms

2 oktober

09:30 hr